

Constraint Satisfaction Problem

Qian Wang

School of Biomedical Engineering
ShanghaiTech University

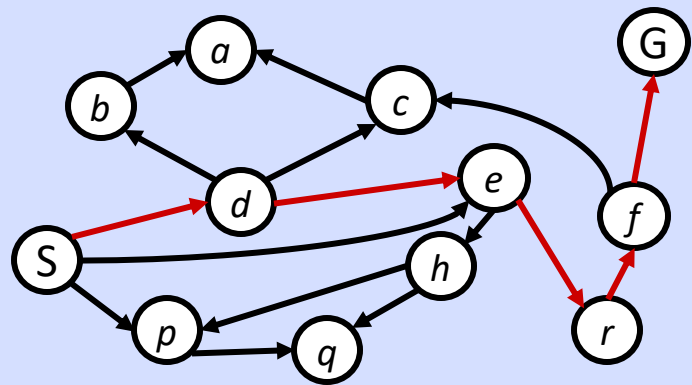
Recap: What is “Search” for?

- Assumption on environment for search problem:
 - Single agent;
 - Deterministic outcomes;
 - Fully observable states;
 - Discrete state and action spaces.
- Search for **planning**
 - Planning a sequence of actions to reach a goal.
 - The path itself is important, not just the goal state.
 - Different paths may have different costs.

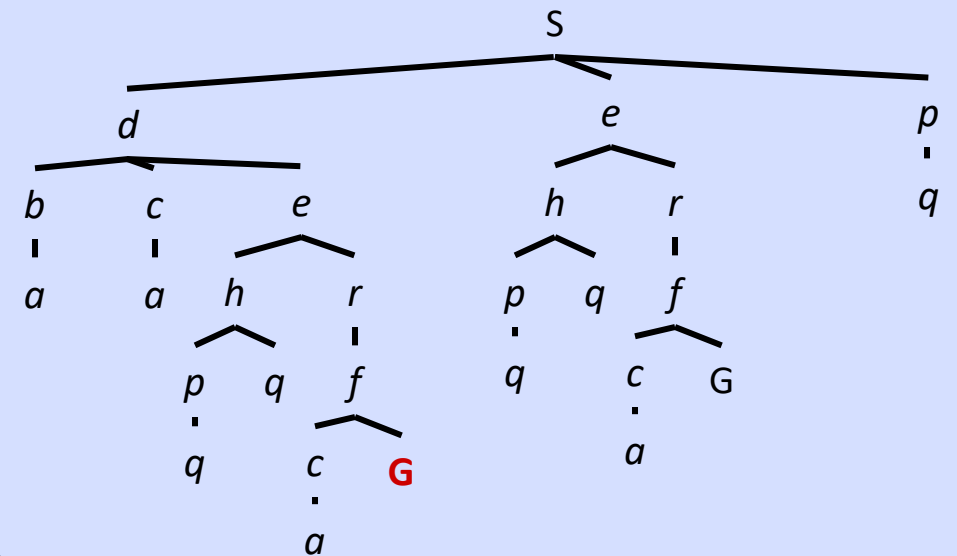


State Space Graph vs. Search Tree

State Space Graph



Search Tree



Search Strategies

- **Uninformed** search:
 - BFS, DFS, IDS, UCS
- **Informed** search:
 - Greedy, A*

Algorithm	Complete?	Optimal?	Time complexity	Space complexity
BFS	Yes	If all step costs are equal	$O(b^s)$	$O(b^s)$
DFS	No	No	$O(b^m)$	$O(bm)$
IDS	Yes	If all step costs are equal	$O(b^s)$	$O(bs)$
UCS	Yes	Yes	Number of nodes with $g(n) \leq C^*$	
Greedy	No	No	Worst case: $O(b^m)$ Best case: $O(bs)$	
A*	Yes	Yes (if heuristic is admissible)	Number of nodes with $g(n)+h(n) \leq C^*$	

What is “Search” for?

- Search for **planning**
- Search for **assignment**
 - The task is to **assign values to variables** while satisfying a set of **constraints**.
 - The focus is on finding a complete and consistent assignment — not the path to the goal, but the final configuration that meets all requirements.



8			4		6			7
						4		
	1					6	5	
5		9		3		7	8	
				7				
	4	8		2		1		3
	5	2					9	
		1						
3			9		2			5

Constraint Satisfaction Problem (CSP)

- A Constraint Satisfaction Problem (CSP) is defined by:
 - **Variables**: each variable X_i represents a component of the state;
 - **Domains**: Each variable X_i takes values from a finite set D_i ;
 - **Constraints**: Rules that specify which combinations of values are allowed for subsets of variables.
- A **solution** to a CSP is an assignment that is both **complete** and **consistent**:
 - Complete assignment: Every variable has been assigned a value;
 - Consistent assignment: No constraints are violated;

Example: Map Coloring

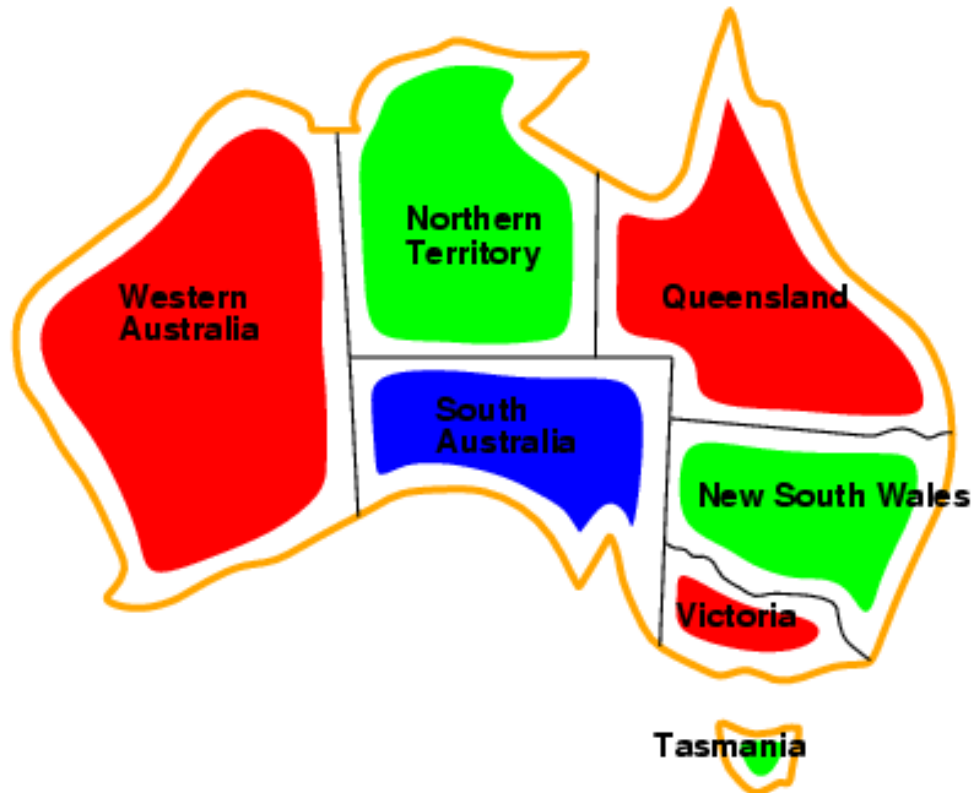
- Variables: WA, NT, Q, NSW, V, SA, T
- Domains: {red, green, blue}
- Constraints: adjacent regions must have different colors



WA $\neq$ NT,
or
(WA, NT) in {(red, green), (red, blue), (green, red),
(green, blue), (blue, red), (blue, green)}

Example: Map Coloring

- Solutions are complete and consistent assignments
 - Example: WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green



The **Four Color Theorem** states that any planar map can be colored using no more than four colors in such a way that no two adjacent regions (i.e., regions that share a border) have the same color.

Example: Sudoku

- Variables: X_{ij}
- Domains: $\{1, 2, \dots, 9\}$
- Constraints:
 Alldiff(X_{ij} in the same *unit*)

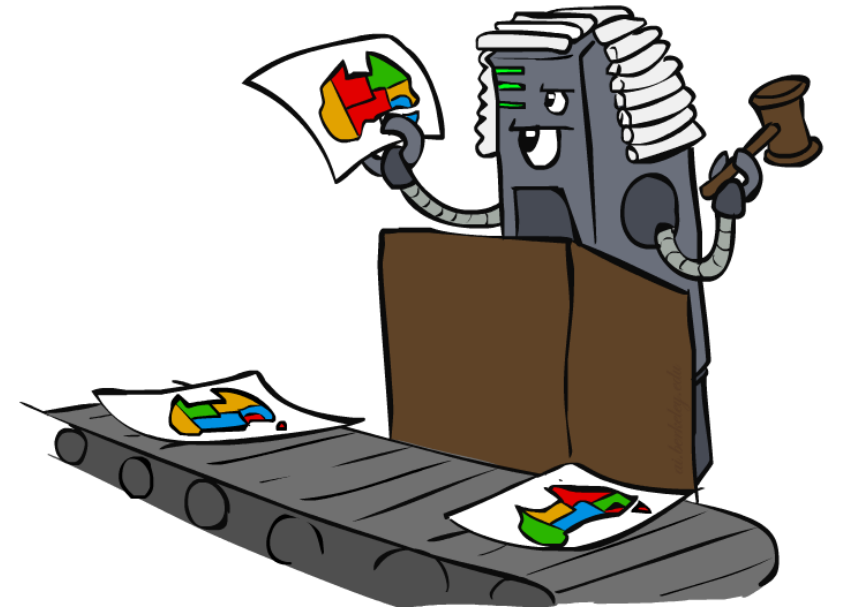
					8			4
	8	4		1	6			
			5			1		
1		3	8			9		
6		8				4		3
		2			9	5	X_{ij}	1
		7			2			
			7	8		2	6	
2			3					

Real-world CSPs

- Assignment problems
 - Example: who teaches what class
- Timetable problems
 - Example: which class is offered when and where
- Transportation scheduling
- Factory scheduling
- More examples of CSPs
 - <http://www.csplib.org/>

CSP as a Search Problem

- We can formulate CSP within the standard search framework:
 - States: Represented by partial assignments;
 - Initial State: The empty assignment;
 - Actions:
 - Select an unassigned variable;
 - Assign it a value that does not violate any constraints;
 - Goal test: The current assignment is complete and consistent.



CSP as a Search Problem

- Depth of any solution:
 - n (assuming there are n variables to assign)
- Total number of possible paths in the search tree:
 - $n! \cdot m^n$ (Each variable can be assigned in any order, and each has m possible values)
- This leads to combinatorial explosion!
 - How can we reduce the branching factor?

Backtracking Search

- In CSP, variable assignments are **commutative**:
 - For example, $[WA = \text{red then } NT = \text{green}]$ is the same as $[NT = \text{green then } WA = \text{red}]$
- Only need to consider one variable assignment per level of the search tree:
 - The order of variable assignments can be fixed;
 - Then there are only m^n leaf nodes, which is much smaller than $n! \cdot m^n$.
- **Backtracking Search**: A depth-first search algorithm tailored for CSP.
 - At each level of the search tree, it assigns a value to one variable at a time;
 - If an assignment violates any constraint, the algorithm backtracks to try a different value or variable.

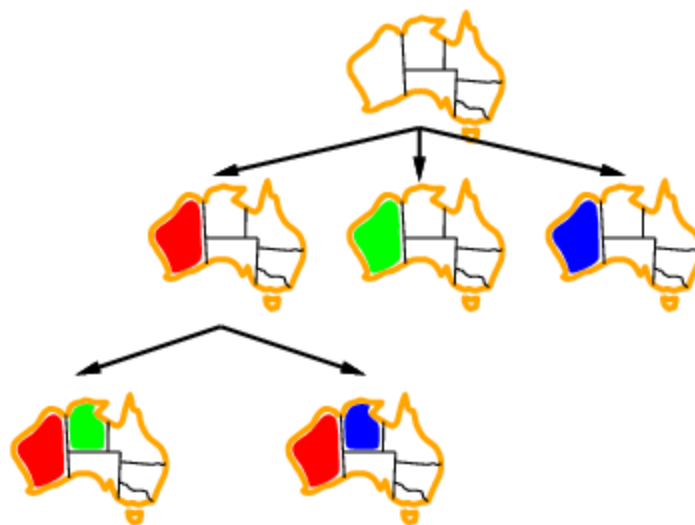
Example



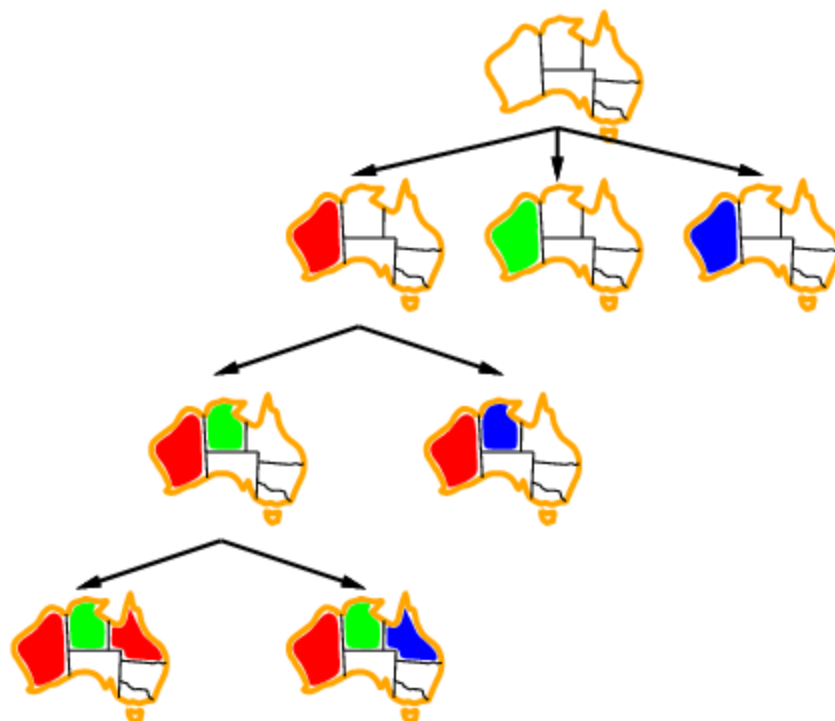
Example



Example



Example



Recursive Backtracking

```
function RECURSIVE-BACKTRACKING(assignment, csp)  
  if assignment is complete then return assignment  
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)  
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp)  
    if value is consistent with assignment given CONSTRAINTS[csp]  
      add {var = value} to assignment  
      result ← RECURSIVE-BACKTRACKING(assignment, csp)  
      if result ≠ failure then return result  
      remove {var = value} from assignment  
  return failure
```

- Ordering: in what order should a **variable** be assigned?
- Ordering: in what order should its **values** be tried?
- Filtering: can we **detect** inevitable failure early?

Which Variable Should be Assigned Next?

- **Minimum remaining values (MRV)** heuristic
 - Choose the most constrained variable, i.e., with the fewest legal values left

Which Variable Should be Assigned Next?

- **Minimum remaining values (MRV) heuristic**
 - Choose the most constrained variable, i.e., with the fewest legal values left

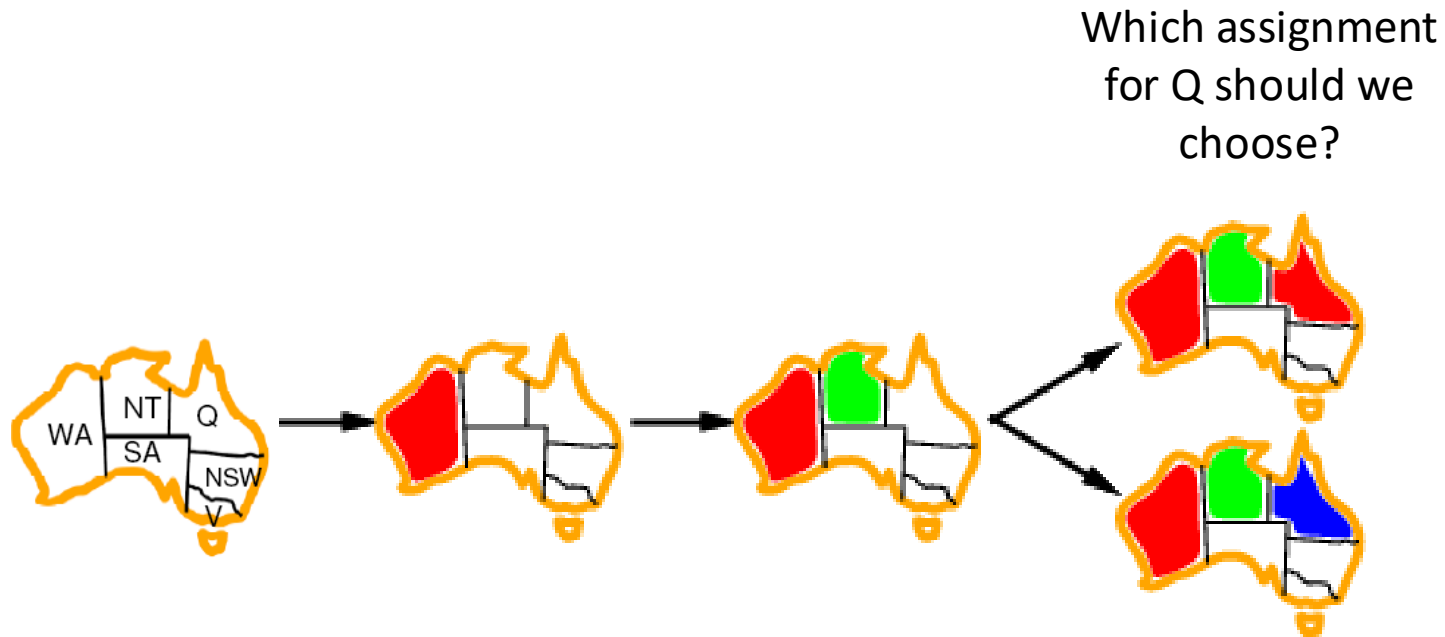


What Value Should be Tried for a Variable?

- Choose the **least constraining value**:
 - The value that rules out the fewest values in the remaining variables

What Value Should be Tried for a Variable?

- Choose the **least constraining value**:
 - The value that rules out the fewest values in the remaining variables



Filtering: Early Detection of Failure

```
function RECURSIVE-BACKTRACKING(assignment, csp)  
  if assignment is complete then return assignment  
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)  
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp)  
    if value is consistent with assignment given CONSTRAINTS[csp]  
      add {var = value} to assignment  
      result ← RECURSIVE-BACKTRACKING(assignment, csp)  
      if result ≠ failure then return result  
      remove {var = value} from assignment  
  return failure
```

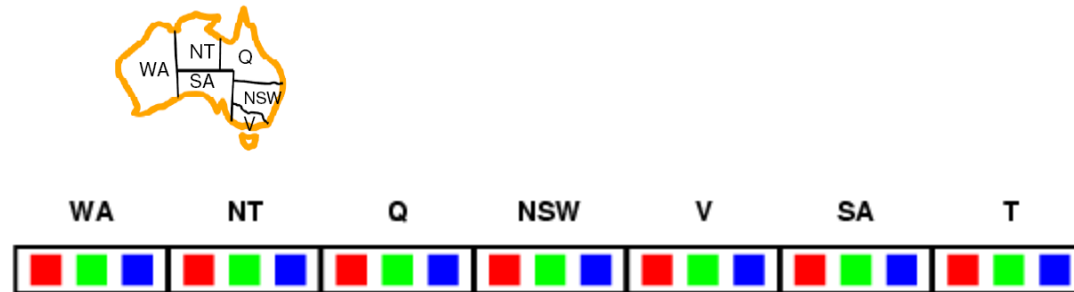


Apply *inference* to reduce the space of possible assignments and detect failure early

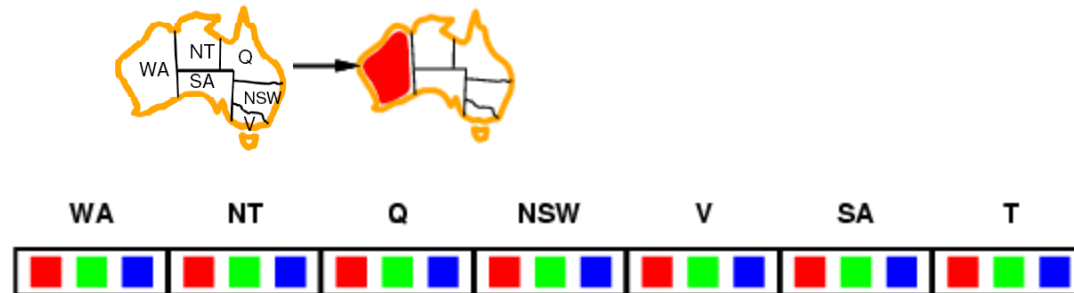
Forward Checking

- To avoid dead ends early, we proactively **check for potential conflicts** during search.
 - After assigning a value to a variable, we track the remaining legal values for all unassigned variables.
 - If any unassigned variable has no legal values left, the search terminates immediately and backtracks.
 - This helps **prune the search space** and prevents exploring paths that cannot lead to a valid solution.

- Forward checking: cross off values that violate a constraint when added to the existing assignment.

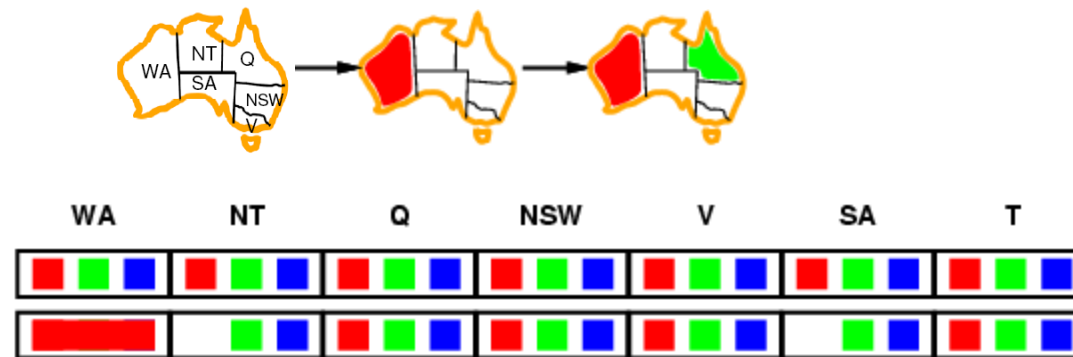


- Forward checking: cross off values that violate a constraint when added to the existing assignment.



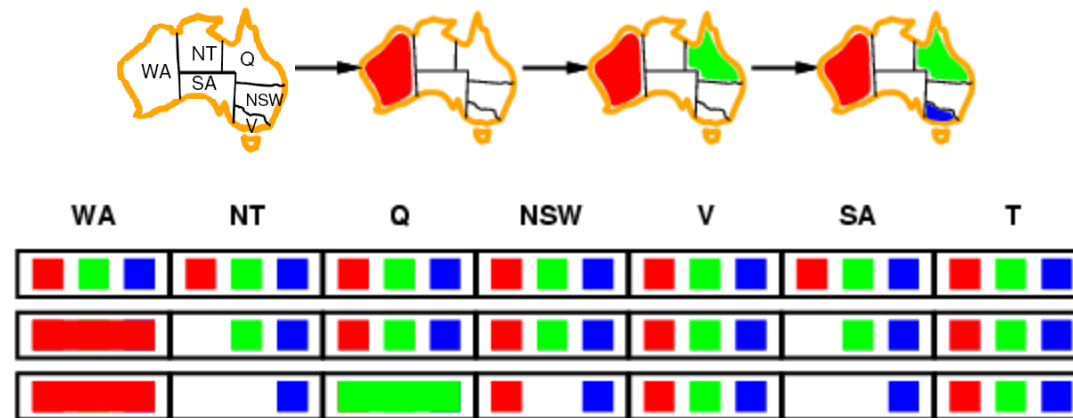
Forward Checking

- Forward checking: cross off values that violate a constraint when added to the existing assignment.



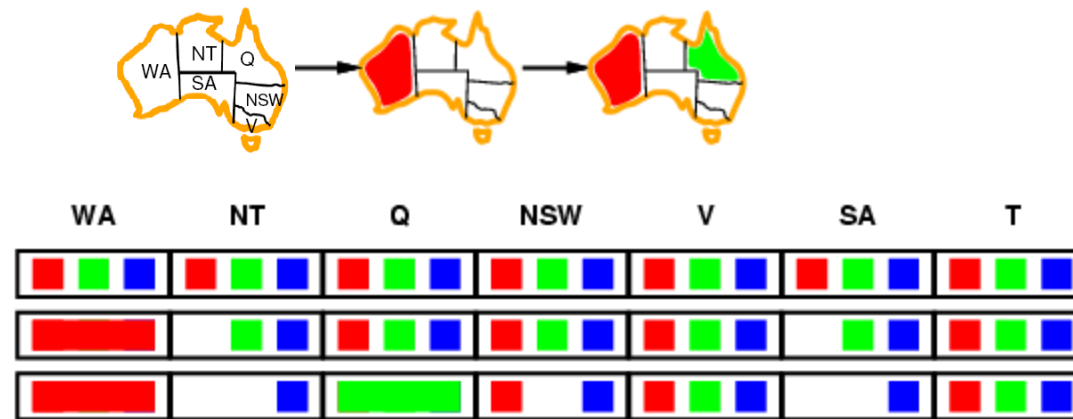
Forward Checking

- Forward checking: cross off values that violate a constraint when added to the existing assignment.



Constraint Propagation

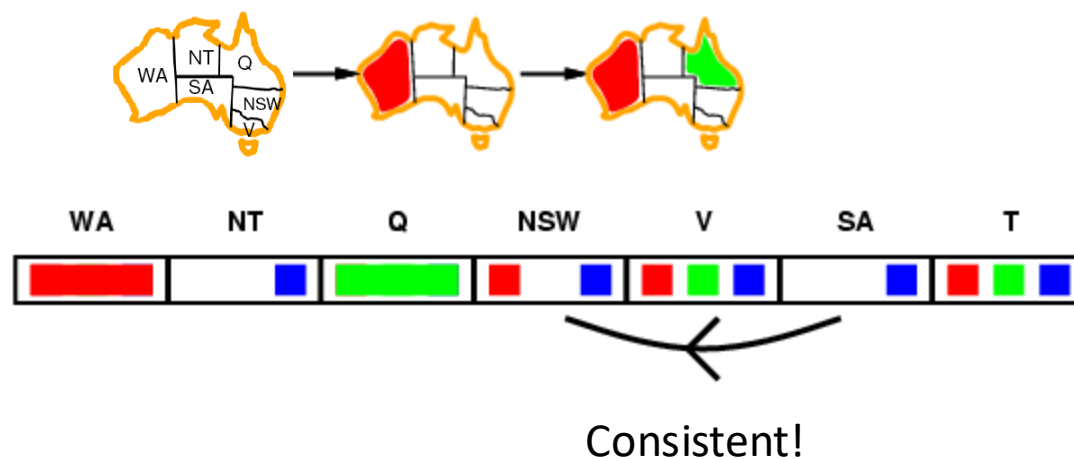
- Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures



- NT and SA cannot both be blue! But why didn't we detect this yet?
- Constraint propagation** repeatedly enforces constraints **locally**.

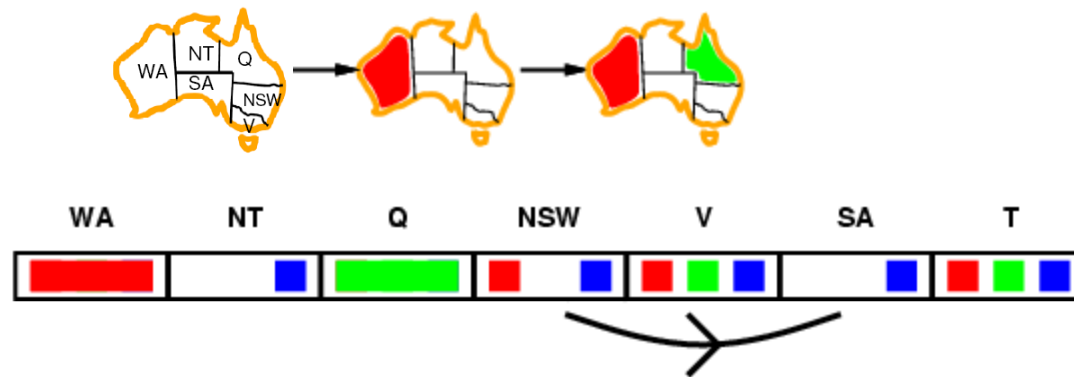
Arc Consistency

- Simplest form of propagation makes each pair of variables consistent:
 - An arc $X \rightarrow Y$ is consistent iff for **every** value of X there is **some** allowed value of Y .



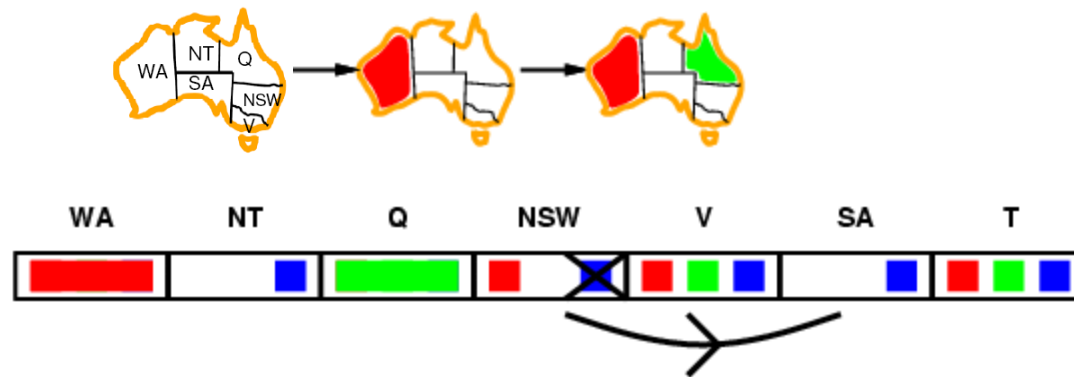
Arc Consistency

- Simplest form of propagation makes each pair of variables consistent:
 - An arc $X \rightarrow Y$ is consistent iff for **every** value of X there is **some** allowed value of Y .



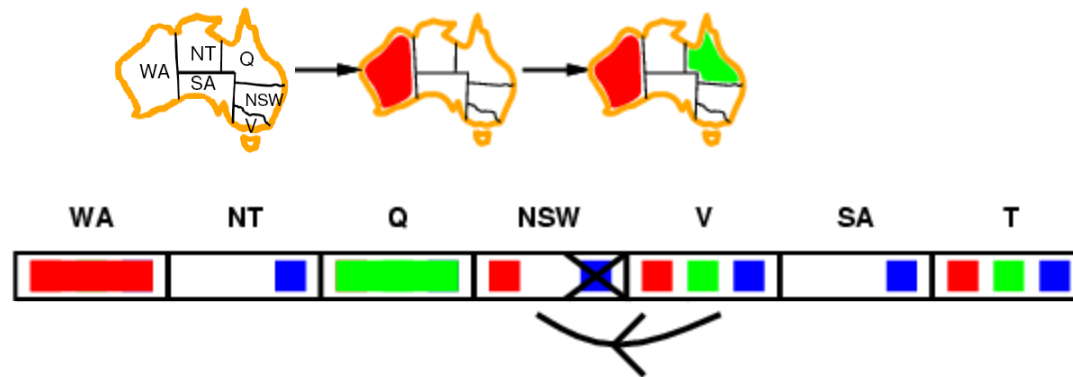
Arc Consistency

- Simplest form of propagation makes each pair of variables consistent:
 - An arc $X \rightarrow Y$ is consistent iff for **every** value of X there is **some** allowed value of Y .
 - When checking $X \rightarrow Y$, throw out any values of X for which there isn't an allowed value of Y .



Arc Consistency

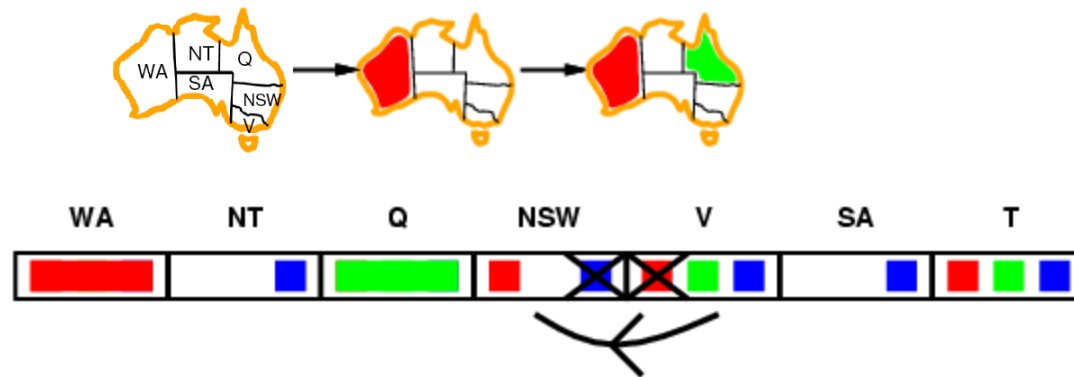
- Simplest form of propagation makes each pair of variables consistent:
 - An arc $X \rightarrow Y$ is consistent iff for **every** value of X there is **some** allowed value of Y .
 - When checking $X \rightarrow Y$, throw out any values of X for which there isn't an allowed value of Y .



- If X loses a value, all pairs $Z \rightarrow X$ need to be rechecked.

Arc Consistency

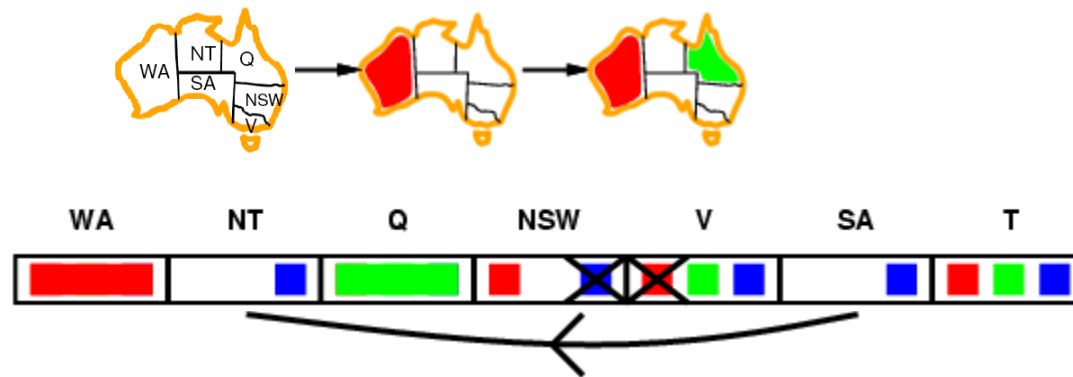
- Simplest form of propagation makes each pair of variables consistent:
 - An arc $X \rightarrow Y$ is consistent iff for **every** value of X there is **some** allowed value of Y .
 - When checking $X \rightarrow Y$, throw out any values of X for which there isn't an allowed value of Y .



- If X loses a value, all pairs $Z \rightarrow X$ need to be rechecked.

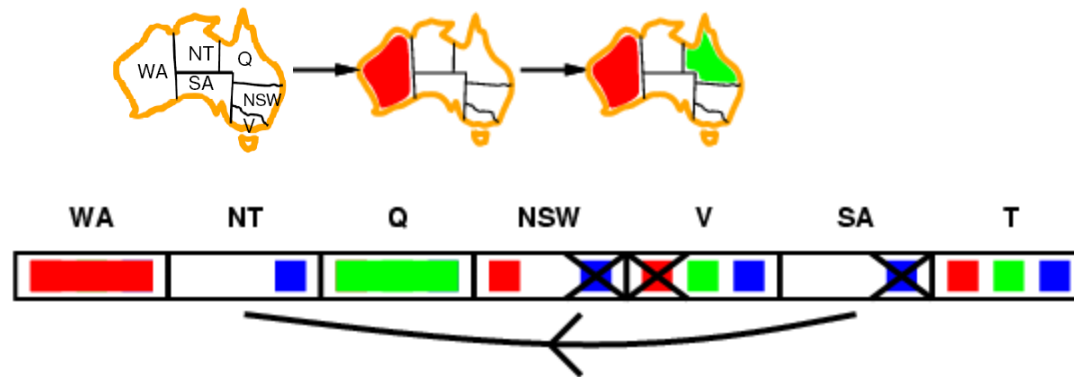
Arc Consistency

- Simplest form of propagation makes each pair of variables consistent:
 - An arc $X \rightarrow Y$ is consistent iff for **every** value of X there is **some** allowed value of Y .
 - When checking $X \rightarrow Y$, throw out any values of X for which there isn't an allowed value of Y .



Arc Consistency

- Simplest form of propagation makes each pair of variables consistent:
 - An arc $X \rightarrow Y$ is consistent iff for **every** value of X there is **some** allowed value of Y .
 - When checking $X \rightarrow Y$, throw out any values of X for which there isn't an allowed value of Y .



- Arc consistency detects failure **earlier** than forward checking.
- Can be run before or after each assignment.

AC-3 Algorithm: Enforcing Arc Consistency

function AC-3(*csp*) **returns** the CSP, possibly with reduced domains

inputs: *csp*, a binary CSP with variables $\{X_1, X_2, \dots, X_n\}$

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\textit{queue})$

if REMOVE-INCONSISTENT-VALUES(X_i, X_j) **then**

for each X_k **in** NEIGHBORS[X_i] **do**

 add (X_k, X_i) to *queue*

function REMOVE-INCONSISTENT-VALUES(X_i, X_j) **returns** true iff succeeds

removed $\leftarrow$ false

for each x **in** DOMAIN[X_i]

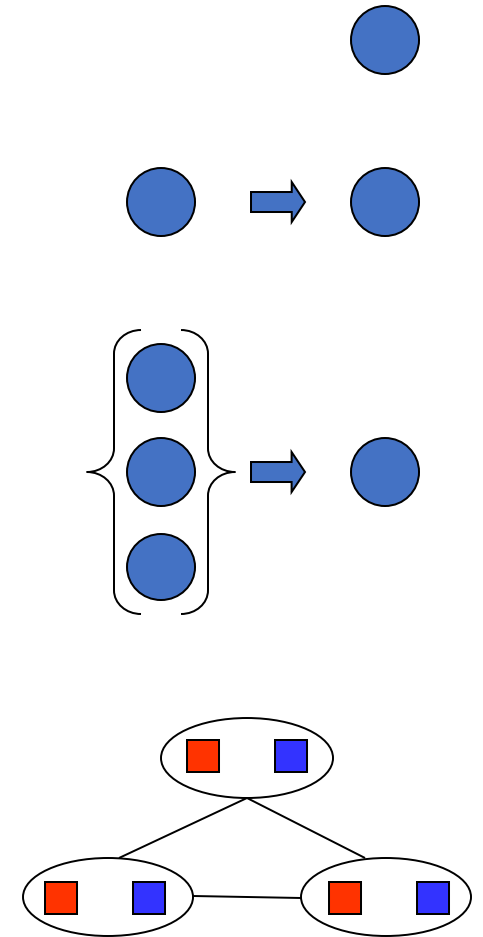
if no value y in DOMAIN[X_j] allows (x, y) to satisfy the constraint $X_i \leftrightarrow X_j$

then delete x from DOMAIN[X_i]; *removed* $\leftarrow$ true

return *removed*

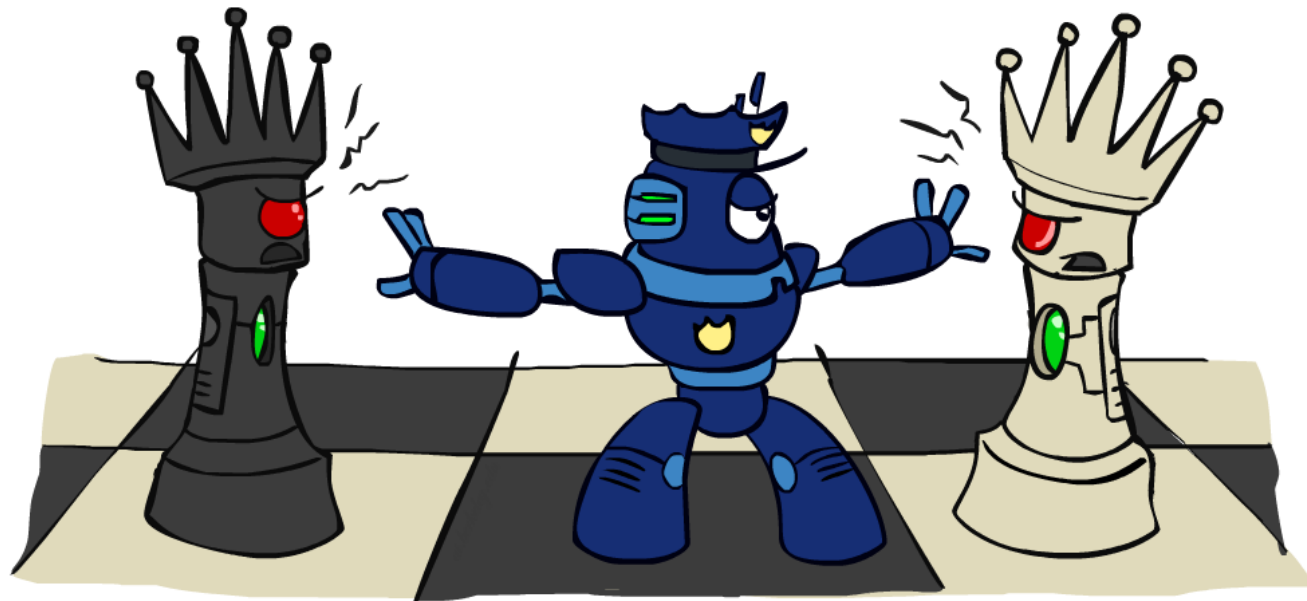
K -Consistency

- Increasing degrees of consistency:
 - 1-Consistency (Node Consistency): Each single node has a value which meets that node's unary constraints;
 - 2-Consistency (Arc Consistency): For each pair of nodes, any consistent assignment to one can be extended to the other;
 - K -Consistency: For each K nodes, any consistent assignment to $k-1$ can be extended to the k -th node.
- Higher k means more expensive computation。



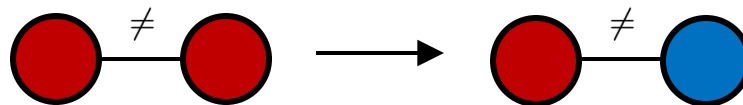
Local Search

- **Local search** offers an alternative to backtracking-style search for solving CSP.
- Instead of building solutions incrementally, it starts with a **complete** assignment (possibly **inconsistent**) and iteratively improves it.



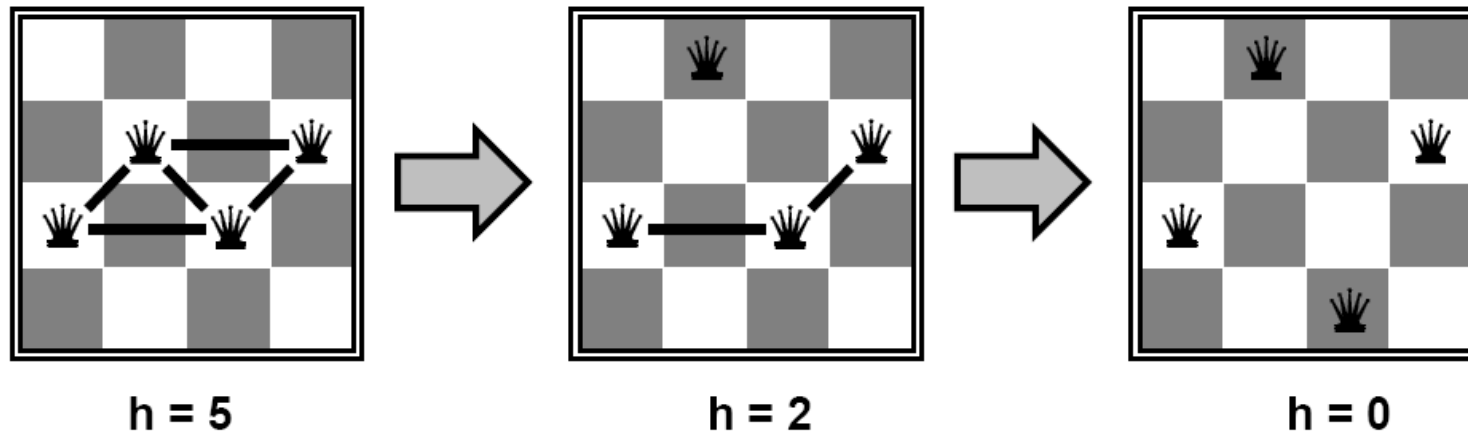
Applying Local Search

- While the assignment is not a solution (i.e., constraints are still violated):
 - **Variable Selection**: Randomly choose a conflicted variable — one involved in constraint violations;
 - **Value Selection** (Min-Conflicts Heuristic): Assign the value that results in the fewest total constraint violations
→ Equivalent to hill climbing with $h(x) = \{\text{number of violated constraints}\}$.



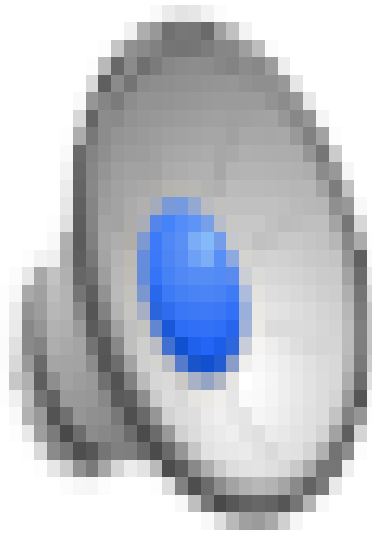
Example: N -Queens ($N=4$)

- States: 4 queens in 4 columns ($4^4 = 256$ states)
- Operators: move queen in column
- Goal test: no conflicts



h = number of conflicts

Example: N -Queens



Summary

- CSPs are special search problems:
 - States: Represented by assignments to a fixed set of variables;
 - Goal Test: Satisfied when all constraints on variable values are met.
- **Backtracking Search**: Depth-first search that assigns one variable at a time and backtracks upon conflict.
 - Variable ordering and value selection both can dramatically improve efficiency;
 - Forward checking prevents assignments that guarantee later failure.
 - Constraint propagation (e.g., arc consistency) does additional work to constrain values and detect inconsistencies.

Thank You!